

## STAT/DATA 1810 · Notes



Week 2 &gt; Fnd 02 · Solution

## Foundations 02 — Data Types and Data Structures

STAT/DATA 1810 · Week 2 Monday · SOLUTION

AUTHOR

Instructor key

```
library(tibble) # today we need tibble() and tribble() – that's it
```

👁️ **Variables check.** Loading `tibble` added *nothing* to the Variables pane. `library()` opens a toolbox; only `<-` creates an object.

**Name:** Instructor key · **Date:** \_\_\_\_\_

Before anything else, type your name and today's date on the line above (and put your name in the `author:` line at the very top of this file).

**How to read this document.** Two labels show up over and over:

- **LM = Learning Moment.** I demonstrate; you follow along and run the code.
- **YT = Your Turn.** You do it yourself — fill the blank or the empty chunk.
- 👁️ **Variables check.** Glance at the **Variables** pane (right side, under SESSION) and answer the question. It is your session's memory made visible.

### Timing & labels

- **Budget:** distinction 4' · types 28' · structures 24' · when things break 6' · activity 12' · exit 5' · save & render 1' = 80'
- **Checkpoint:** §2 should start by the **32' mark** — if behind, drop YT4 and activity items E/F
- **LM Learning Moment** — instructor demonstrates, students follow
- **YT Your Turn** — students do it themselves
- 👁️ **Variables check** — the Fnd 01 / Lab 01 habit; keep it alive: every 👁️ is a 20-second look at the pane, not a discussion
- Say the LM/YT names out loud the first time each appears — students otherwise guess
- Practice goes to **vectors, data frames/tibbles, lists**; matrix & tribble get one chunk each, no drills
- **Packages:** only `tibble`. Say it out loud: “*we load what today needs, not the whole tidyverse*”

0 · The one distinction

4 min

**Type** = what a single value *is* (number? text? TRUE/FALSE?). **Structure** = the container holding many values (vector? data frame? list?).

Everything today hangs off that sentence.

You already know how to insert a chunk (**Cmd/Ctrl + Shift + I**, from last week). Whenever you see “**+ insert a chunk**”, make one — every YT today depends on it.

Teaching note — the sentence on the board

- Write it and leave it there all class: **type = one value’s nature; structure = the container**
- Chunk insertion was taught in Foundations 01 — do **not** re-teach it; if someone is stuck, a neighbor shows them the shortcut
- One line on the `setup` chunk: we load `tibble` only. Last week’s `library(tidyverse)` loads nine packages; today needs one function family from one of them

1 · Data types

28 min

1.1 · The types, defined first (before any code)

5 min

Every single value in R has a **type**. Here are the ones you’ll live with, each with a Cal Poly example:

| Type             | What it is                                       | Example value                                |
|------------------|--------------------------------------------------|----------------------------------------------|
| numeric (double) | a number that can have decimals                  | 4.75 — a latte at Julian’s                   |
| integer          | a whole number (written with an <code>L</code> ) | 38L — people in the Rec Center at 7 am       |
| character        | text, in quotes                                  | "Scout Coffee"                               |
| logical          | TRUE or FALSE                                    | TRUE — the drink is iced                     |
| factor           | text from a <i>fixed menu</i> of allowed values  | "morning" from {morning, afternoon, evening} |
| Date / date-time | a calendar date; a date <i>and</i> clock time    | 2026-09-22 / 2026-09-22 07:00:00             |
| NA               | a missing value — a hole, not a type of its own  | the headcount nobody recorded                |

Teaching note — definitions before class()

- Read the table row by row *before* anyone runs code. The order today is: **define the type** → **then discover it with `class()`**
- Ask: “If I write a latte price, which row?” (numeric) “The name of the shop?” (character) “Whether it’s iced?” (logical)
- **NA** gets one sentence: it’s a placeholder for “nobody knows,” and it can sit inside any type

## 1.2 · Finding the type with `class()` 7 min

**LM1 — `class()` tells you the type (4 min).** Predict each `class()` from the table above, *then* run:

```
# LM1: predict from the table, then run
class(4.75)           # Julian's latte
```

```
[1] "numeric"
```

```
class(38L)           # people in the Rec Center at 7 am
```

```
[1] "integer"
```

```
class("Scout Coffee") # where the latte came from
```

```
[1] "character"
```

```
class(TRUE)          # iced?
```

```
[1] "logical"
```

*Q1. Predictions: numeric · integer · character · logical*

### Expected — LM1 & Q1

- **LM1** `"numeric" · "integer" · "character" · "logical"`
- **Q1** most students predict `"numeric"` for `38L` — the `L` suffix is worth 30 seconds, no more: R stores whole numbers as integers when you ask, otherwise as doubles

**LM2 — doubles vs integers (1 min).** Both are numeric; `typeof()` tells them apart.

```
# LM2
typeof(4.75)
```

```
[1] "double"
```

```
typeof(38L)
```

```
[1] "integer"
```

LM3 — dates and date-times (2 min).

```
# LM3
today <- Sys.Date()
class(today)
```

```
[1] "Date"
```

```
right_now <- Sys.time()
class(right_now)
```

```
[1] "POSIXct" "POSIXt"
```

- Expected — LM3
- LM3 "Date" and "POSIXct" "POSIXt"
  - Say “date-time” and move on — lubridate handles the ugliness in week 4

1.3 · Three families: quantitative, categorical, other 6 min

R’s types sort into three families you’ll use to pick plots and summaries all term:

| Family            | Question it answers                                | R types                    | Cal Poly examples                                                                  |
|-------------------|----------------------------------------------------|----------------------------|------------------------------------------------------------------------------------|
| Quantitative      | <i>how much? how many?</i><br>— you can average it | numeric, integer           | latte price (4.75), Rec Center headcount (38), miles from campus (2.3)             |
| Categorical       | <i>which kind? which group?</i><br>— you count it  | character, factor, logical | coffee shop ("Julian's", "Starbucks", "Scout"), time slot ("morning"), iced (TRUE) |
| Other (time/date) | <i>when?</i> — ordered, but not a plain number     | Date, date-time            | the day you bought the latte, the clock time you swiped into the Rec               |

## What kind of variable is it?

Ask the question in the header — the answer picks the family, the family picks the plot

| QUANTITATIVE<br>"how much? how many?"                                                                                                                                      | CATEGORICAL<br>"which kind? which group?"                                                                                                                                                           | OTHER · time & date<br>"when?"                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>You can AVERAGE it</b></p> <ul style="list-style-type: none"> <li>latte price — 4.75</li> <li>Rec Center headcount — 38L</li> <li>miles from campus — 2.3</li> </ul> | <p><b>You COUNT it, not average it</b></p> <ul style="list-style-type: none"> <li>coffee shop — "Scout Coffee"</li> <li>time slot — "morning"</li> <li>iced? — TRUE (a 2-level category)</li> </ul> | <p><b>Ordered, but not a plain number</b></p> <ul style="list-style-type: none"> <li>day you bought the latte — 2026-09-22</li> <li>Rec swipe-in time — 07:00:00</li> </ul> |
| <p><b>R TYPES</b></p> <p>numeric integer</p> <p>glimpse tags: &lt;dbl&gt; &lt;int&gt;</p>                                                                                  | <p><b>R TYPES</b></p> <p>character factor logical</p> <p>glimpse tags: &lt;chr&gt; &lt;fct&gt; &lt;lgl&gt;</p>                                                                                      | <p><b>R TYPES</b></p> <p>Date date-time</p> <p>glimpse tags: &lt;date&gt; &lt;dtm&gt;</p>                                                                                   |

### Inside CATEGORICAL: character vs factor — same words, different powers

#### character — free text

any string is welcome: names, comments, typos too

"morning" "Morning" "AM" "morning "

→ four DIFFERENT groups · tables sort alphabetically

use for: names, free-form answers, anything typed

#### factor — text with a fixed menu (levels), in YOUR order

levels = c("morning", "afternoon", "evening")

morning afternoon evening "Morning"?

→ not on the menu → becomes NA — silently, no error

use for: categories with a known set · plots & tables follow YOUR level order

The three families and, inside categorical, the character-vs-factor split. The dashed "Morning"? chip is the trap: a value not on a factor's menu becomes NA with no error.

Q2. Which family: (a) the number of people in the Rec Center at 5 pm · (b) the shop that made your latte · (c) the date of the game vs. UCSB · (d) whether you bought a latte today (yes/no)?

Your answer: (a) quantitative · (b) categorical · (c) other — a date · (d) categorical — a logical is a two-level category.

### Q2 answer + why the families matter

- Q2 (a) **quantitative** (b) **categorical** (c) **other / date** (d) **categorical** — logical is a yes/no category
- Why bother: quantitative → histograms and means; categorical → bar charts and counts; dates → line plots over time. Week 3's plots pick themselves once you name the family
- Trap to name now: a number can be categorical. Your student ID is digits, but averaging student IDs is nonsense

**A small distinction — "string" vs "character".** Other languages call text a *string*. R calls the *type* **character**, and one piece of text ("Scout Coffee") is a **string** — a character value of length one. So "latte" is a string, its class is character, and c("latte", "mocha") is a character *vector* of two strings. You'll hear both words; they are not in conflict.

## 1.4 · Factors vs characters — why it matters 6 min

**LM4 — the same text, two types (4 min).** Rec Center headcounts come with a time slot. As character, the slots are just words; as a **factor**, they come with a fixed menu of allowed values (**levels**) in an order

you choose:

```
# LM4: character vs factor
slot_chr <- c("evening", "morning", "afternoon", "morning", "evening", "evening")
slot_fct <- factor(slot_chr, levels = c("morning", "afternoon", "evening"))

class(slot_chr)
```

```
[1] "character"
```

```
class(slot_fct)
```

```
[1] "factor"
```

```
levels(slot_fct)
```

```
[1] "morning"  "afternoon" "evening"
```

```
table(slot_chr)  # alphabetical – afternoon first
```

```
slot_chr
afternoon  evening  morning
         1         3         2
```

```
table(slot_fct)  # the order YOU chose – morning first
```

```
slot_fct
morning afternoon  evening
         2         1         3
```

👁️ **Variables check.** `slot_chr` and `slot_fct` are both listed — same six values, but Variables describes one as *character* and one as *factor with 3 levels*. That description IS the type.

*Q3. How many levels does `slot_fct` have, and what order did we give them? Why would that order matter on a bar chart?*

*Your answer:* 3 levels — morning, afternoon, evening. A bar chart follows the levels, so the factor plots the day in time order; the character version plots alphabetically (afternoon, evening, morning), which reads as nonsense.

### Q3 answer — the two reasons factors exist

- **Q3 3 levels**, in the order morning → afternoon → evening
- **Reason 1 — order.** Characters sort alphabetically; factors sort by *your* levels. Every plot and table of a categorical variable inherits that order

- **Reason 2 — a fixed menu.** A factor knows "night" is *not* allowed (it becomes NA); a character column will happily accept "Morning", "morning ", and "AM" as three different groups. Factors are how you catch typos in categories
- Rule of thumb to leave them with: **character = free text (names, comments); factor = a category with a known set of values**
- The menu metaphor pays rent in week 6 when forcats arrives

**LM5 — NA, the missing value (2 min).** `airquality` (built in) has real missing ozone readings; `is.na()` finds them and `sum()` counts them:

```
# LM5: count the missing values in Ozone
sum(is.na(airquality$Ozone))
```

```
[1] 37
```

*Q4. How many Ozone values are missing? 37*

#### Q4 answer

- **Q4 37 missing**
- First appearance of the `is.na()` + `sum()` idiom; students use it in every cycle

**LM6 — the is.\* family (2 min).** Test types with functions that return TRUE/FALSE:

```
# LM6: any correct fills work, e.g.
is.numeric(4.75)
```

```
[1] TRUE
```

```
is.character("Scout Coffee")
```

```
[1] TRUE
```

```
is.logical(TRUE)
```

```
[1] TRUE
```

```
is.factor(slot_fct)
```

```
[1] TRUE
```

*YT1 (4 min). + Insert a chunk below. In it, find the `class()` of each of the following, predicting first: "TRUE" (with quotes) · `4.75 > 4.25` · `mtcars$cyl` · `Sys.Date()` + 7. Then in the comment, write which one surprised you.*

```
# YT1 – surprise: "TRUE" in quotes is character, and a comparison makes a logical
class("TRUE")
```

```
[1] "character"
```

```
class(4.75 > 4.25)
```

```
[1] "logical"
```

```
class(mtcars$cyl)
```

```
[1] "numeric"
```

```
class(Sys.Date() + 7)
```

```
[1] "Date"
```

*Predictions: character · logical · numeric · Date*

#### Expected — YT1

- **YT1** "character" · "logical" · "numeric" · "Date"
- Surprise #1: "TRUE" — quotes make it text (a string!)
- Surprise #2: 4.75 > 4.25 produces a logical — comparisons are how logicals get made in practice (`filter()` in two weeks)
- `mtcars$cyl` being numeric though it only takes 4/6/8 previews activity item E: **the type R stores is not always the family the data means**

## 2 · Data structures 24 min

**LM7 — vector: one type, many values (4 min).** Built with `c()`. Three places to get a latte near campus:

```
# LM7: a vector
shops <- c("Julian's", "Starbucks", "Scout Coffee")
shops
```

```
[1] "Julian's"      "Starbucks"     "Scout Coffee"
```

```
length(shops)
```

```
[1] 3
```

```
class(shops)
```

```
[1] "character"
```

YT2 (4 min). ➕ Insert a chunk. Build THREE vectors and print each: (a) *price* — what a latte costs at each shop above (Julian's on campus, the Starbucks in the UU, Scout downtown — use real prices if you know them, e.g. 4.75, 5.45, 5.95); (b) *size* — the size you'd order at each, as text ("small", "medium", "large"); (c) *iced* — three TRUE/FALSE values, whether you'd get it iced. Then run *class()* on each. What happens if you run *c(4.75, "five", 5.95)*? Try it and look at the class.

```
# YT2
price <- c(4.75, 5.45, 5.95)
size  <- c("small", "medium", "large")
iced  <- c(TRUE, FALSE, TRUE)
price; size; iced
```

```
[1] 4.75 5.45 5.95
```

```
[1] "small" "medium" "large"
```

```
[1] TRUE FALSE TRUE
```

```
class(price); class(size); class(iced)
```

```
[1] "numeric"
```

```
[1] "character"
```

```
[1] "logical"
```

```
c(4.75, "five", 5.95)
```

```
[1] "4.75" "five" "5.95"
```

```
class(c(4.75, "five", 5.95))
```

```
[1] "character"
```

Q5. *c(4.75, "five", 5.95)* has class character. In one phrase, why?

Your answer: A vector holds one type, so R coerces everything to text.

**Q5 answer — coercion**

- **Q5 character**
- A vector holds *one* type, so R converts everything to the type that can hold all of it — text
- This is **coercion**, arriving by accident rather than lecture — say the word once
- They meet it again when a price column imports as `<chr>` because one row says “free”

**LM8 — data frame & tibble: vectors standing side by side (6 min).** Each column is a vector; every column has the same length.

```
# LM8: same data, two containers
latte_df <- data.frame(shop = shops, price = c(4.75, 5.45, 5.95))
latte_tbl <- tibble(shop = shops, price = c(4.75, 5.45, 5.95))

latte_df
```

| shop         | price |
|--------------|-------|
| <chr>        | <dbl> |
| Julian's     | 4.75  |
| Starbucks    | 5.45  |
| Scout Coffee | 5.95  |
| 3 rows       |       |

```
latte_tbl
```

| shop         | price |
|--------------|-------|
| <chr>        | <dbl> |
| Julian's     | 4.75  |
| Starbucks    | 5.45  |
| Scout Coffee | 5.95  |
| 3 rows       |       |

*Q6. Name one visible difference between how the data frame and the tibble printed:*

*Your answer:* The tibble announces its dimensions (`# A tibble: 3 × 2`) and shows column types (`<chr>`, `<dbl>`); the data frame shows neither.

#### Q6 model answers (any one)

- **Q6** the tibble announces its dimensions (`# A tibble: 3 × 2`)
- the tibble shows column types (`<chr>`, `<dbl>`) — **the families from §1.3, printed for you**
- the data frame shows row numbers without types
- **Framing:** a tibble IS a data frame with better manners — we'll say “data frame” for both

*YT3 (5 min). + Insert a chunk. Build a tibble called `lattes` from YOUR vectors from YT2 — columns `shop`, `price`, `size`, `iced`. Print it. Then run `nrow(lattes)`, `ncol(lattes)`, and*

`class(lattes)`. What happens if one vector has only two values? Try it (make a broken one on purpose), read the error, then fix it.

```
# YT3
lattes <- tibble(shop = shops, price = price, size = size, iced = iced)
lattes
```

| shop         | price | size   | iced  |
|--------------|-------|--------|-------|
| <chr>        | <dbl> | <chr>  | <lgl> |
| Julian's     | 4.75  | small  | TRUE  |
| Starbucks    | 5.45  | medium | FALSE |
| Scout Coffee | 5.95  | large  | TRUE  |
| 3 rows       |       |        |       |

```
nrow(lattes); ncol(lattes); class(lattes)
```

```
[1] 3
```

```
[1] 4
```

```
[1] "tbl_df"      "tbl"        "data.frame"
```

👁️ **Variables check.** `lattes` gets an expand arrow that `price` did not. Click it: four columns, each with its own type and length 3. A data frame is a container of vectors — and the pane shows you exactly that.

```
# YT3 – the deliberate break
tibble(shop = shops, price = c(4.75, 5.45))
#> Error: Tibble columns must have compatible sizes.
#> • Size 3: Existing data.
#> • Size 2: Column `price`.
```

```
# YT3 – the fix: give price all three values
tibble(shop = shops, price = c(4.75, 5.45, 5.95))
```

| shop         | price |
|--------------|-------|
| <chr>        | <dbl> |
| Julian's     | 4.75  |
| Starbucks    | 5.45  |
| Scout Coffee | 5.95  |
| 3 rows       |       |

**YT3 teaching note — a good error**

- **YT3** The error names the sizes *and* the column — have someone read it aloud
- “Every column the same length” is now something they’ve felt, not heard
- `class(lattes)` returns three things (`tbl_df`, `tbl`, `data.frame`) — say “it’s a data frame, with extra labels” and move on

**LM9 — tribble: a tibble typed row by row, with a factor inside (3 min).** Rec Center headcounts, three slots. Note the `slot` column is made a factor *with the order we want*:

```
# LM9: Rec Center headcounts — a tribble, then a factor column
rec <- tribble(
  ~slot,      ~hour, ~people,
  "morning",   7,    38,
  "afternoon", 13,   112,
  "evening",   18,   164
)
rec$slot <- factor(rec$slot, levels = c("morning", "afternoon", "evening"))
rec
```

| slot      | hour  | people |
|-----------|-------|--------|
| <fct>     | <dbl> | <dbl>  |
| morning   | 7     | 38     |
| afternoon | 13    | 112    |
| evening   | 18    | 164    |
| 3 rows    |       |        |

**LM9 — read the type tags**

- `slot` prints as `<fct>`, `hour` and `people` as `<dbl>` — categorical, quantitative, quantitative. The tibble is telling you the families
- `rec$slot <- factor(...)` is the first time they *change* a column’s type — one sentence: “we told R this text is a category with an order”

**LM10 — list: the junk drawer (4 min).** A list holds *anything*, even other structures:

```
# LM10
my_list <- list(name = "week 2", nums = c(1, 2, 3), data = latte_tbl)
my_list
```

```
$name
[1] "week 2"

$nums
[1] 1 2 3

$data
```

```
# A tibble: 3 × 2
  shop      price
<chr>    <dbl>
1 Julian's  4.75
2 Starbucks 5.45
3 Scout Coffee 5.95
```

```
length(my_list)
```

```
[1] 3
```

### LM10 — junk-drawer framing

- **LM10** A list holds anything, including a whole tibble
- `length()` is 3 — three drawers, whatever's inside them
- Plant the flag: “when we pull data from an API in week 8, it arrives as a list”

YT4 (3 min). **+** Insert a chunk. Build a list called `me` with three named parts: your first name (text), the number of classes you're taking this quarter, and your `lattes` tibble. Print it. Notice how each part keeps its own type.

```
# YT4 — e.g.
me <- list(first = "Eman", n_classes = 4, lattes = lattes)
me
```

```
$first
[1] "Eman"

$n_classes
[1] 4

$lattes
# A tibble: 3 × 4
  shop      price size iced
<chr>    <dbl> <chr> <lgl>
1 Julian's  4.75 small TRUE
2 Starbucks 5.45 medium FALSE
3 Scout Coffee 5.95 large TRUE
```

### LM11 — matrix: all one type, rows × columns (2 min) (brief):

```
# LM11
matrix(1:6, nrow = 2)
```

```
 [,1] [,2] [,3]
[1,]  1   3   5
[2,]  2   4   6
```

Q7. Which structure could hold a column of shop names AND a column of prices: a matrix or a data frame? Why?

Your answer: A data frame — a matrix forces every cell to one type, so the prices would be coerced to text.

#### Q7 answer

- **Q7 a data frame**
- A matrix forces every cell to one type, so names + prices would coerce the prices to text — same coercion as Q5, now at the structure level

## 2b · When things break 6 min

Today's errors are *type* and *structure* errors. Same rule as last week: read the message slowly, name the family, then fix it.

YT5 (6 min). Each chunk below has ONE bug. Read the error (or the silent wrong answer!), fix it in place, and write the error family in the comment.

```
# YT5a – error family: ____
price <- c(4.75, "5.45", 5.95)
mean(price)
```

```
# YT5a – error family: coercion – one quoted number turned the whole vector to
  character; mean() warns and returns NA
price <- c(4.75, 5.45, 5.95)
mean(price)
```

```
[1] 5.383333
```

```
# YT5b – error family: ____
tibble(shop = c("Julian's", "Starbucks", "Scout Coffee"), price = c(4.75, 5.45))
```

```
# YT5b – error family: column lengths don't match (size 3 vs size 2) – every column
  must be the same length
tibble(shop = c("Julian's", "Starbucks", "Scout Coffee"), price = c(4.75, 5.45,
  5.95))
```

| shop      | price |
|-----------|-------|
| <chr>     | <dbl> |
| Julian's  | 4.75  |
| Starbucks | 5.45  |

| shop         | price |
|--------------|-------|
| <chr>        | <dbl> |
| Scout Coffee | 5.95  |

3 rows

```
# YT5c – error family: ____
factor(c("morning", "Morning", "evening"), levels = c("morning", "afternoon",
  "evening"))
```

```
# YT5c – error family: a value not on the factor's menu becomes NA (no error!) –
  "Morning" ≠ "morning"
factor(c("morning", "morning", "evening"), levels = c("morning", "afternoon",
  "evening"))
```

```
[1] morning morning evening
Levels: morning afternoon evening
```

### YT5 answers — three error families for types & structures

- **YT5a** `c(4.75, "5.45", 5.95)` → the vector is **character**; `mean()` warns “argument is not numeric or logical: returning NA”
  - family: **coercion** — one text value drags the whole vector to text. The tell is the quotes in the printout
  - fix: remove the quotes (later: `as.numeric()`)
- **YT5b** **Tibble columns must have compatible sizes — size 3 vs size 2**
  - family: **mismatched column lengths** — the error names the sizes *and* the column
  - fix: give every column the same number of values
- **YT5c** no error at all — **"Morning"** prints as **<NA>**
  - family: **value not in the factor's levels** → silently becomes **NA** (the sneakiest of the three; the menu metaphor from §1.4 explains it)
  - fix: match the level exactly (**"morning"**) — R is case-sensitive
- Two of the three are *silent*: no red text. Say it: **the absence of an error is not the presence of a right answer** — print your objects

## 3 · In-class activity: type it or contain it 12 min

For each item, write **T** if the question is about a type and **S** if it's about a structure, then answer it with code. **+** Insert **one chunk per item** (A through F) so each answer is its own runnable piece.

- **A.** What is `rec` (from LM9) — its structure?
- **B.** What is `rec$people` — its type? Which family?
- **C.** Is there any NA anywhere in `airquality$Solar.R`? How many?
- **D.** Build a tibble with 2 columns: three places you eat near campus and a rating (1–10) for each.

- **E.** What is `airquality$Month` — its type? Should it be? (Months are stored as numbers here — would a factor make more sense? One sentence.)
- **F.** Build a list holding your food tibble from D *and* a character vector of what you order at each place.

#### Teaching note — one chunk per item

- Check that students actually made **six chunks**
- The habit of “one idea, one chunk” is what makes ETV documents readable

```
# A – Structure
class(rec)          # a tibble (tbl_df / tbl / data.frame)
```

```
[1] "tbl_df"      "tbl"        "data.frame"
```

```
# B – Type
class(rec$people)   # "numeric" – quantitative
```

```
[1] "numeric"
```

```
# C – Type question (is.na tests values)
sum(is.na(airquality$Solar.R))  # 7
```

```
[1] 7
```

```
# D – Structure built by the student, e.g.
eats <- tibble(
  place = c("Woodstock's", "Firestone", "Campus Market"),
  rating = c(9, 10, 6)
)
eats
```

| place         | rating |
|---------------|--------|
| <chr>         | <dbl>  |
| Woodstock's   | 9      |
| Firestone     | 10     |
| Campus Market | 6      |
| 3 rows        |        |

```
# E – Type; stored as integer, arguably should be a factor (or a Date)
class(airquality$Month)
```

```
[1] "integer"
```

```
# F – Structure: a list holding a tibble and a vector
eats_list <- list(
  places = eats,
  orders = c("pizza", "tri-tip sandwich", "burrito")
)
eats_list
```

```
$places
# A tibble: 3 × 2
  place      rating
  <chr>      <dbl>
1 Woodstock's      9
2 Firestone       10
3 Campus Market    6

$orders
[1] "pizza"          "tri-tip sandwich" "burrito"
```

*A: S · B: T (numeric, quantitative) · C: T (7 missing) · D: S · E: T — integer; a factor would make more sense since 5–9 are labels, not amounts · F: S*

#### Activity answers

- **A** = S (structure — a tibble)
- **B** = T — "numeric", quantitative
- **C** = T — 7 missing
- **D** = S
- **E** = T — "integer"; a factor would make sense since 5–9 are labels, not amounts
- **F** = S
- **E is the discussion item:** *the type R stores is not always the family the data means* — that's the whole reason `mutate()` + `factor()` exists, and it's §1.4 again

## 4 · Exit question 5 min

*In your own words, one sentence each: what is a data type, and what is a data structure? Use one example of each from today — and at least one must be something YOU built.*

*Your answer:* A data type is what one value is — 4.75 (a Julian's latte) is numeric. A data structure is the container holding values — my lattes tibble is a data frame whose iced column is logical.

#### Model answer

- "A data type is what one value is — 4.75 is numeric. A data structure is the container holding values — my lattes tibble is a data frame whose iced column is logical."

- Passes: any answer pairing one correct **type** example with one correct **structure** example
- At least one example should be from a student-built object
- Bonus signal: a student who names the *family* (quantitative/categorical) alongside the type has the whole lesson

## 5 · Save & render 1 min

Save this file into your [R/Notes/](#) folder as `fnd02_yourlastname.qmd`. Then **Restart R** (top-right session menu → Restart) — watch Variables empty out — and press **Render** (Cmd/Ctrl + Shift + K). If a chunk you inserted breaks the render, read the error: it names the chunk. Restarting first is how you prove that everything the document needs is *in* the document.

### Teaching note — restart, then render

- Same close as Fnd 01 and Lab 01: **restart** → **Variables empties** → **render**. The 10-second rebuild is the argument for code-in-the-file
- A render that fails on a student-inserted chunk is a *good* failure: the error names the chunk — have them read it aloud